

COMPLETE SQL JOINS GUIDE — ALL TYPES

INNER, LEFT, RIGHT, FULL OUTER, SELF & CROSS JOIN — Using Your Live college_demo Database

Every query below is verified against your actual data — run them yourself and confirm the outputs match

1. Your Current Schema (Recap)

This document uses your exact live tables, as shown by your DESCRIBE output. Nothing here is hypothetical - every query and result below reflects your actual database right now.

Table	Key Columns
department	dept_id (PK), dept_name (UNIQUE)
student	roll_no (PK), name, email (UNIQUE), aadhar_no (UNIQUE), dept_id (FK), cgpa
course	course_id (PK), course_name, dept_id (FK)
enrollment	roll_no (PK,FK), course_id (PK,FK), semester (PK), grade

Your Current Data

department:

dept_id	dept_name
1	Computer Science
2	Electronics

student (8 rows):

roll_no	name	dept_id	cgpa
101	Nilisha	1	9.0
102	Rahul	2	8.0
104	neha	1	8.9
106	nehas	1	9.9
107	test 5	1	2.0
201	test1	2	7.0
202	test2	2	7.2
203	test3	1	6.8

course:

course_id	course_name	dept_id
501	DBMS	1
502	Circuits	2

enrollment (only 2 rows - important for later):

roll_no	course_id	semester	grade
101	501	3	A
101	502	3	B

IMPORTANT: Notice: only roll_no 101 (Nilisha) has any enrollment records at all. The other 7 students exist in the student table but have never enrolled in anything. Keep this in mind - it drives most of the examples in this document.

What This Document Covers

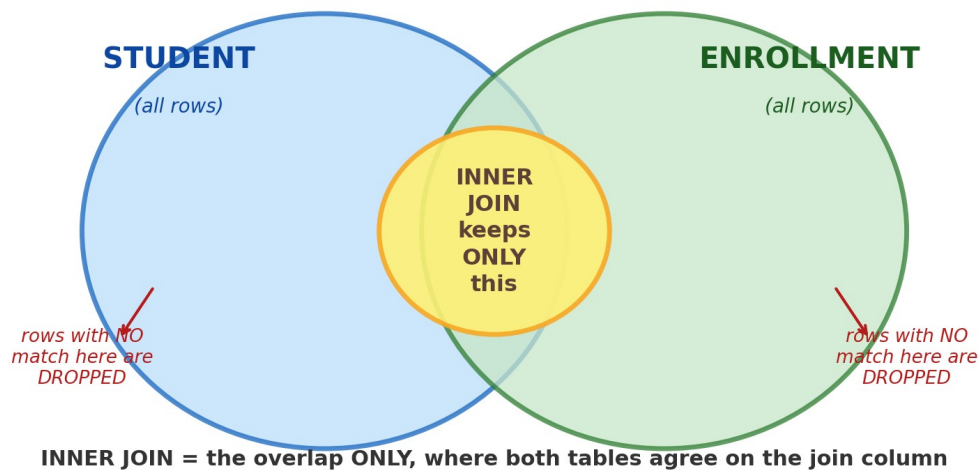
Section	Join Type
2-10	INNER JOIN (definition, aliases, 6 worked examples, pitfalls)
11	LEFT JOIN
12	RIGHT JOIN
13	FULL OUTER JOIN
14	SELF JOIN
15	CROSS JOIN
16-17	Practice exercises and summary

2. What INNER JOIN Actually Does

INNER JOIN combines rows from two tables, keeping ONLY the rows where the join condition finds a match on BOTH sides. If a row on either side has no matching partner, it is silently dropped from the result entirely - not shown as NULL, not shown with blanks, just excluded.

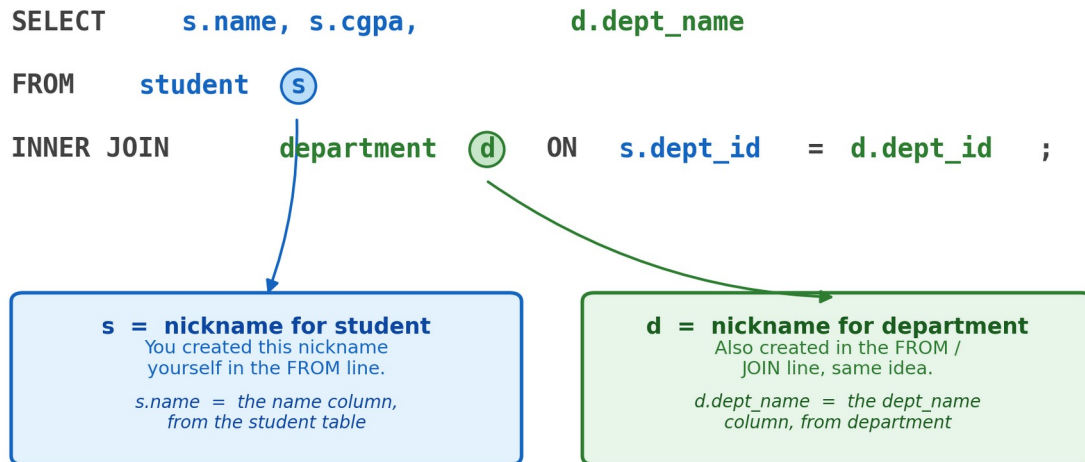
```
SELECT columns  
FROM table1  
INNER JOIN table2 ON table1.column = table2.column;
```

Writing just JOIN (without the word INNER) means exactly the same thing - INNER is the default join type and is often left out.



2b. Understanding Table Aliases (the s. and d. you'll see everywhere)

Before the examples, one thing needs to be crystal clear: what are s, d, e, and c doing in front of column names throughout this document? This single diagram answers it.



Rule: `table_alias . column_name` — tells SQL exactly which table a column comes from

REMEMBER: Aliases are not special syntax to memorize - they are nicknames YOU invent, right there in the FROM/JOIN line. Once invented, every column from that table gets referred to with `alias.column_name` for the rest of the query.

3. Example 1 — student INNER JOIN department

Goal: show each student's name alongside their department name.

```
SELECT s.name, s.cgpa, d.dept_name
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id;
```

What happens step by step:

- MySQL takes every row in student
- For each one, it looks for a department row where dept_id matches
- Every student in your table has dept_id 1 or 2, and BOTH exist in department - so all 8 students find a match

Verified Output (8 rows):

name	cgpa	dept_name
Nilisha	9.0	Computer Science
Rahul	8.0	Electronics
neha	8.9	Computer Science
nehas	9.9	Computer Science
test 5	2.0	Computer Science
test1	7.0	Electronics
test2	7.2	Electronics
test3	6.8	Computer Science

NOTE: Since every student's dept_id happens to exist in the department table, this INNER JOIN loses no rows at all - the result has exactly 8 rows, same as the student table alone. This will NOT be true in the next example.

4. Example 2 — student INNER JOIN enrollment (the critical lesson)

Goal: show each student's name alongside the courses they're enrolled in.

```
SELECT s.name, e.course_id, e.grade
FROM student s
INNER JOIN enrollment e ON s.roll_no = e.roll_no;
```

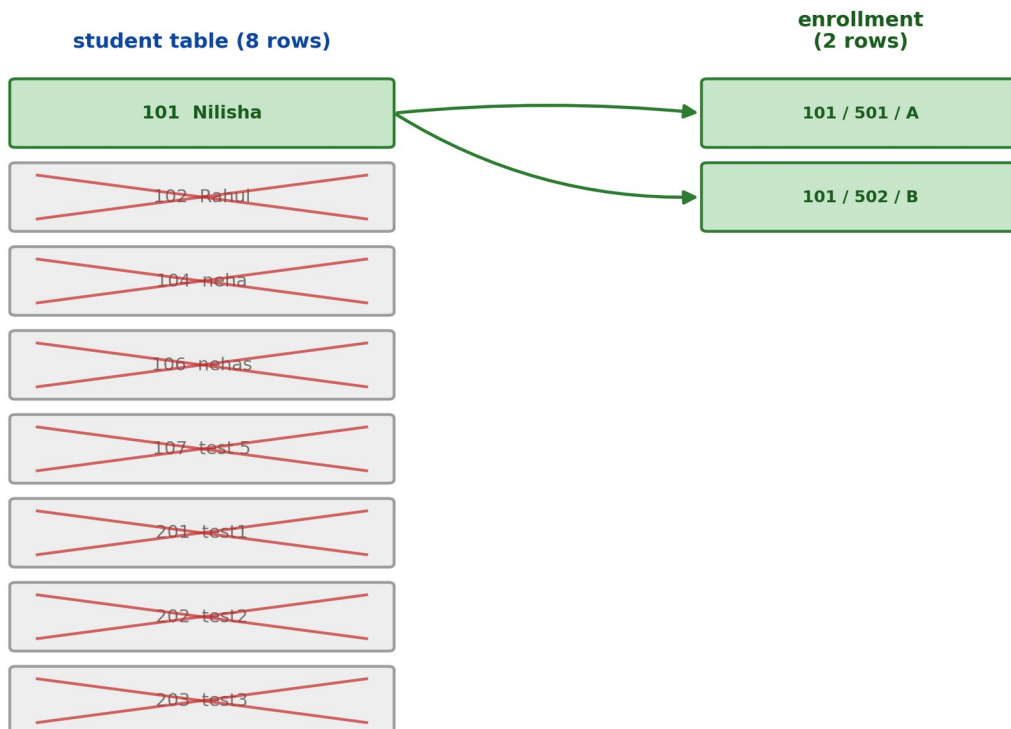
What happens step by step:

- MySQL takes every row in student (8 rows)
- For each one, it looks for enrollment rows where roll_no matches
- Only roll_no 101 exists in the enrollment table - the other 7 students (102, 104, 106, 107, 201, 202, 203) have NO matching row
- INNER JOIN silently drops all 7 unmatched students - they do not appear in the output at all

Verified Output (only 2 rows, not 8):

name	course_id	grade
Nilisha	501	A
Nilisha	502	B

Here is exactly what happened to all 8 rows, visually:



RESULT: only 2 rows survive (both Nilisha) — the other 7 students vanish completely

KEY LESSON: This is the single most important thing to internalize about INNER JOIN: it is not 'student data plus enrollment info if any exists' - it is 'only the rows where BOTH sides have a match.' If you actually wanted to see all 8

students, with blanks for those who haven't enrolled in anything, you would need a LEFT JOIN instead (covered in the full Joins Deep Dive document).

5. Example 3 — Three-Table Join: student + enrollment + course

Goal: show student name, course name, and grade together - requires chaining two INNER JOINS.

```
SELECT s.name, c.course_name, e.grade
FROM student s
INNER JOIN enrollment e ON s.roll_no = e.roll_no
INNER JOIN course c ON e.course_id = c.course_id;
```

What happens step by step:

- First, student INNER JOIN enrollment narrows things down to just Nilisha's 2 rows, exactly as in Example 2
- Then, that intermediate result is joined again with course, matching course_id to fetch the course name
- Both course_id 501 and 502 exist in the course table, so both of Nilisha's rows survive this second join too

Verified Output (2 rows):

name	course_name	grade
Nilisha	DBMS	A
Nilisha	Circuits	B

Notice the row count stayed at 2 through both joins - once INNER JOIN has narrowed the result down, each additional INNER JOIN can only keep the same rows or shrink further, never grow back the students that were already excluded.

6. Example 4 — All Four Tables Joined Together

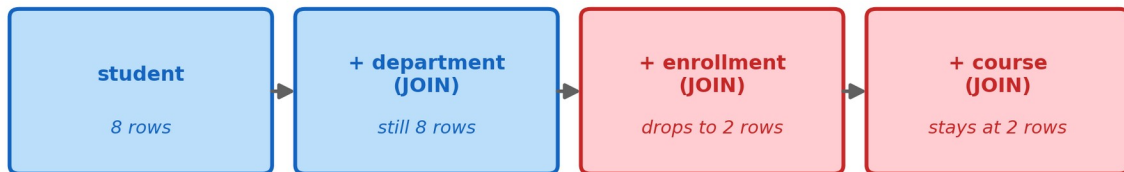
Goal: the complete picture - student, their department, their course, and their grade, all in one row.

```
SELECT s.name, d.dept_name, c.course_name, e.grade
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id
INNER JOIN enrollment e ON s.roll_no = e.roll_no
INNER JOIN course c ON e.course_id = c.course_id;
```

Verified Output (still 2 rows):

name	dept_name	course_name	grade
Nilisha	Computer Science	DBMS	A
Nilisha	Computer Science	Circuits	B

Here's the row count at each step of the chain:



The chain is only ever as wide as its NARROWEST join — once enrollment drops it to 2 rows, no later JOIN can bring the other 6 students back

NOTE: Even though department INNER JOIN would have kept all 8 students on its own (Example 1), the moment it's chained with the enrollment join, the whole chain is limited by the SMALLEST surviving set - which is enrollment's 2 rows. Order of the JOIN clauses doesn't change this outcome, only which columns are available at each step.

7. Example 5 — INNER JOIN with a WHERE Filter

Goal: find only students with cgpa above 8, along with their department name.

```
SELECT s.name, s.cgpa, d.dept_name
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id
WHERE s.cgpa > 8;
```

Checking against your real data:

Nilisha (9.0), neha (8.9), and nehas (9.9) all exceed 8.0. Rahul (8.0) does NOT count - the condition is strictly greater than 8, not 8 or above. test5 (2.0), test1 (7.0), test2 (7.2), and test3 (6.8) are all below the threshold too.

Verified Output (3 rows):

name	cgpa	dept_name
Nilisha	9.0	Computer Science
neha	8.9	Computer Science
nehas	9.9	Computer Science

The WHERE clause is applied AFTER the join has combined matching rows - so it's filtering the already-joined result, not either table individually.

8. Example 6 — INNER JOIN with Aggregate Functions

Goal: count students per department, and find the average CGPA per department.

6a. Student count per department

```
SELECT d.dept_name, COUNT(*) AS student_count
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Computer Science has dept_id 1: Nilisha, neha, nehas, test5, test3 = 5 students. Electronics has dept_id 2: Rahul, test1, test2 = 3 students.

dept_name	student_count
Computer Science	5
Electronics	3

6b. Average CGPA per department

```
SELECT d.dept_name, AVG(s.cgpa) AS avg_cgpa
FROM student s
INNER JOIN department d ON s.dept_id = d.dept_id
GROUP BY d.dept_name;
```

Computer Science: $(9.0 + 8.9 + 9.9 + 2.0 + 6.8) / 5 = 36.6 / 5 = 7.32$. Electronics: $(8.0 + 7.0 + 7.2) / 3 = 22.2 / 3 = 7.40$.

dept_name	avg_cgpa
Computer Science	7.32
Electronics	7.40

OBSERVATION: test5's very low cgpa of 2.0 is dragging the Computer Science average down noticeably compared to Electronics, despite CS also having the two highest individual scores (nehas at 9.9, Nilisha at 9.0). This is a good real example of why averages alone can hide a lot of spread in the data.

9. A Shortcut: the USING Clause

When the join column has the EXACT SAME NAME in both tables (as dept_id does here, in both student and department), you can use USING instead of writing out the full ON condition.

```
-- These two queries are identical:  
SELECT s.name, d.dept_name FROM student s  
JOIN department d ON s.dept_id = d.dept_id;  
  
SELECT s.name, d.dept_name FROM student s  
JOIN department d USING (dept_id);
```

USING is purely a shorthand - it only works when the column name matches exactly on both sides, and it's a matter of style preference whether to use it.

10. A Pitfall Specific to Your Schema: Ambiguous Columns

Your schema has dept_id appearing in THREE different tables: student, course, and department. If you join more than one of these together and reference dept_id without specifying which table it comes from, MySQL will refuse to guess.

```
-- THIS WILL FAIL with an ambiguous column error:
SELECT name, dept_id FROM student
JOIN department ON student.dept_id = department.dept_id;
-- Error: Column 'dept_id' in field list is ambiguous
```

The fix is always to prefix the column with its table (or alias):

```
-- CORRECT:
SELECT s.name, s.dept_id FROM student s
JOIN department d ON s.dept_id = d.dept_id;
```

WATCH OUT: This isn't a hypothetical warning - your actual schema has this exact ambiguity built in across student, course, and department. Get in the habit of aliasing every table (s, d, c, e as used throughout this document) and prefixing every shared column name from the start.

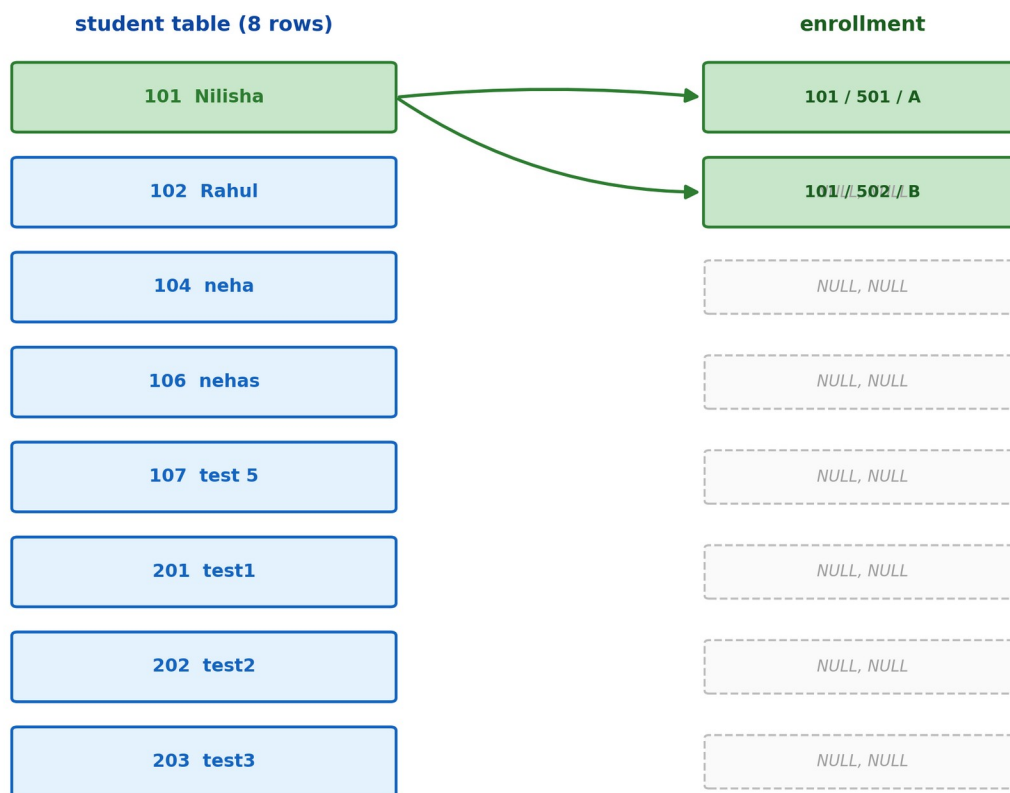
11. LEFT JOIN — Keep All Students, Even Without Enrollments

Goal: list every student, showing their enrollment info where it exists - and NULL where it doesn't.

```
SELECT s.name, e.course_id, e.grade
FROM student s
LEFT JOIN enrollment e ON s.roll_no = e.roll_no;
```

What happens step by step:

- Every one of the 8 student rows is guaranteed to appear, no matter what
- Where a match exists in enrollment (only roll_no 101), the matching columns are filled in
- Where no match exists (the other 7 students), course_id and grade simply show as NULL - the student row is still kept



RESULT: ALL 9 rows kept (8 students + 1 extra for Nilisha's 2nd course)
Unmatched students show NULL instead of vanishing

Verified Output (9 rows total):

name	course_id	grade
Nilisha	501	A
Nilisha	502	B
Rahul	NULL	NULL
neha	NULL	NULL

name	course_id	grade
nehas	NULL	NULL
test 5	NULL	NULL
test1	NULL	NULL
test2	NULL	NULL
test3	NULL	NULL

NOTE: Notice the row count is 9, not 8 - Nilisha appears TWICE because she has two enrollment rows, and LEFT JOIN still produces one output row per match. This is different from a plain student list, where you'd see her only once.

The Most Useful LEFT JOIN Pattern: Finding What's Missing

Add one WHERE clause and this same LEFT JOIN becomes the standard way to find students with NO enrollments at all:

```
SELECT s.name
FROM student s
LEFT JOIN enrollment e ON s.roll_no = e.roll_no
WHERE e.roll_no IS NULL;
```

Against your data, this returns exactly the 7 students without any enrollment: Rahul, neha, nehas, test 5, test1, test2, and test3.

12. RIGHT JOIN — The Mirror Image of LEFT JOIN

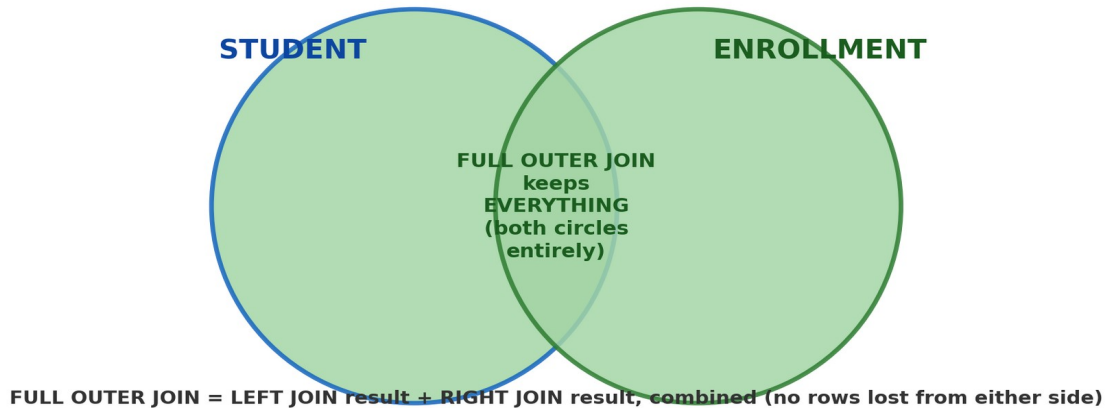
Goal: identical result to the LEFT JOIN above, but written with the tables in the opposite order.

```
SELECT s.name, e.course_id, e.grade
FROM enrollment e
RIGHT JOIN student s ON e.roll_no = s.roll_no;
```

KEY INSIGHT: This produces the EXACT SAME 9 rows as the LEFT JOIN example in Section 11. RIGHT JOIN keeps everything from the table on the right side (student here) - swap the table order and use LEFT JOIN instead, and you get an identical result. This is why most developers rarely use RIGHT JOIN in practice: it's easier to always default to LEFT JOIN and just order tables accordingly.

13. FULL OUTER JOIN — Keep Absolutely Everything

Goal: show every student AND every enrollment record, matched where possible, with NULLs on whichever side has no match.



IMPORTANT: MySQL does NOT support FULL OUTER JOIN directly as a keyword. It must be simulated using UNION of a LEFT JOIN and a RIGHT JOIN.

```
SELECT s.name, e.course_id, e.grade
FROM student s LEFT JOIN enrollment e ON s.roll_no = e.roll_no
UNION
SELECT s.name, e.course_id, e.grade
FROM student s RIGHT JOIN enrollment e ON s.roll_no = e.roll_no;
```

What this returns on your actual data:

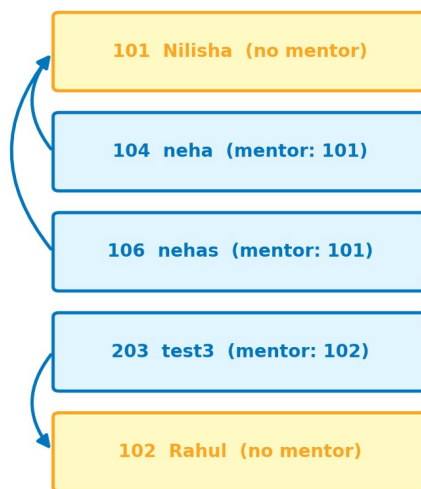
Since every single row in your enrollment table (both belong to roll_no 101) already has a matching student, there are no 'orphan' enrollment rows pointing to a nonexistent student. That means on YOUR current data, this FULL OUTER JOIN produces the exact same 9 rows as the plain LEFT JOIN in Section 11 - nothing extra shows up from the RIGHT side.

WHY THIS MATTERS: This won't always be true. If you inserted an enrollment row for a roll_no that doesn't exist in student (which your foreign key constraint would actually block, since enrollment.roll_no references student.roll_no), THAT orphan row is exactly what FULL OUTER JOIN would add beyond what LEFT JOIN alone shows. Your foreign key constraint from Session 2 is precisely what makes this scenario impossible here - a good example of constraints and JOIN behavior connecting back to earlier material.

14. SELF JOIN — A Table Joined to Itself

Your current schema has no self-referencing column (nothing like a `mentor_roll_no` or `manager_id`), so there's no live self-join example to run directly. Here's how it would work if you added one - this is the standard pattern for representing hierarchies (employees and managers, students and mentors, categories and subcategories).

HYPOTHETICAL: if student had a `mentor_roll_no` column



self-join: `student e JOIN student m ON e.mentor_roll_no = m.roll_no`
(same table, two aliases: `e` = mentee, `m` = mentor)

```
-- Hypothetical: ALTER TABLE student ADD mentor_roll_no INT;

SELECT e.name AS mentee, m.name AS mentor
FROM student e
JOIN student m ON e.mentor_roll_no = m.roll_no;
```

Why This Needs Two Aliases

SQL cannot join a table to itself without giving it two different names - `e` (for the mentee's row) and `m` (for the mentor's row) let SQL treat the same physical table as if it were two separate tables for the duration of this one query.

15. CROSS JOIN — Every Possible Combination

Goal: demonstrate what happens with NO join condition at all - every student paired with every course.

```
SELECT s.name, c.course_name
FROM student s
CROSS JOIN course c;
```

student (8 rows) × course (2 rows) = 16 combinations

	DBMS	Circuits
Nilisha	Nilisha + DBMS	Nilisha + Circuits
Rahul	Rahul + DBMS	Rahul + Circuits
neha	neha + DBMS	neha + Circuits
nehas	nehas + DBMS	nehas + Circuits
test 5	test 5 + DBMS	test 5 + Circuits
test1	test1 + DBMS	test1 + Circuits
test2	test2 + DBMS	test2 + Circuits
test3	test3 + DBMS	test3 + Circuits

Every cell = one output row. No condition links student to course at all - it's pure multiplication.

Verified Row Count on Your Data

8 students x 2 courses = 16 rows total - every student paired with both DBMS and Circuits, regardless of whether they're actually enrolled in either.

CAUTION: Compare this to Example 2's INNER JOIN, which correctly returned only 2 real enrollments. CROSS JOIN ignores the enrollment table's actual relationships entirely - it's rarely what you want by accident. The most common real cause of an ACCIDENTAL cross join is forgetting the ON condition, or writing FROM student, course (a comma-separated list) instead of an explicit JOIN.

16. Practice Exercises (Try These Yourself)

Write and run each query against your actual database, then check your row count and values against the hints given.

Exercise 1

Write a query joining course and department to show each course's name alongside its department name. (Hint: expect exactly 2 rows - DBMS/Computer Science and Circuits/Electronics.)

Exercise 2

Write a query joining student and enrollment and course together to show name, course_name, semester, and grade. (Hint: expect exactly 2 rows, both for Nilisha - re-derive this from Example 3's logic plus the course table.)

Exercise 3

Write a query using INNER JOIN and WHERE to find students in Electronics (dept_id 2) with a cgpa below 7.5. (Hint: check Rahul, test1, and test2 against this condition individually.)

Exercise 4

Predict, without running it: if you INSERT a new enrollment row for roll_no 102 (Rahul) into a course, and re-run Example 2's query, how many rows would the result have? Then actually insert one and confirm your prediction.

Exercise 5 — LEFT JOIN

Write a LEFT JOIN between department and course to confirm every department has at least one course. (Hint: since both departments already have exactly one course each, no NULLs should appear.)

Exercise 6 — CROSS JOIN row count

If you added one more course to the course table (making it 3 total), how many rows would the CROSS JOIN in Section 15 produce? Add the course and confirm.

17. Summary — What to Remember About Every Join Type

- **INNER JOIN keeps ONLY rows with a match on both sides - unmatched rows vanish completely.**
- LEFT JOIN keeps everything from the left table, filling in NULL where the right table has no match.
- RIGHT JOIN is LEFT JOIN with the tables swapped - most teams standardize on LEFT for consistency.
- FULL OUTER JOIN keeps everything from both tables - MySQL requires simulating it via UNION of LEFT and RIGHT.
- SELF JOIN joins a table to itself using two aliases - needed whenever a table has a hierarchical relationship to itself.
- CROSS JOIN produces every possible combination with no condition at all - almost always a mistake unless deliberately intended.
- Chaining multiple JOINS means the final row count is bounded by whichever table in the chain has the fewest matches.
- Always alias your tables and prefix shared column names to avoid ambiguous column errors - your schema's dept_id makes this a real, not hypothetical, concern.